CS336B — Object Oriented Programming with Java

Mid Semester Examination – 1 (Blended ODD SEM 2026–27)
Complete Solutions for Questions 1–8

Complete exam-ready solutions based on the uploaded examination paper.

Q1. Spacecraft Fuel Calculation

Question: Create a Spacecraft class with distance and fuelEfficiency. calculateFuel() should calculate distance / fuelEfficiency and handle fuelEfficiency = 0 using try-catch, displaying "Invalid fuel efficiency!".

Complete Java Answer

```
public class Main {
    public static void main(String[] args) {
        Spacecraft s = new Spacecraft();
        s.distance = 10;
        s.fuelEfficiency = 0;
        s.calculateFuel();
    }
}

class Spacecraft {
    double distance;
    double fuelEfficiency;

    void calculateFuel() {
        try {
            if (fuelEfficiency == 0) {
                throw new ArithmeticException();
            }

            double requiredFuel = distance / fuelEfficiency;
            System.out.println("Required Fuel: " + requiredFuel);
        }
        catch (ArithmeticException e) {
            System.out.println("Invalid fuel efficiency!");
        }
    }
}
```

Key points: Uses a class, data members, a method, explicit exception throwing, and try-catch. The explicit zero check is used because double division by zero normally produces Infinity.

Output:

```
Invalid fuel efficiency!
```

Q2. Artifact Tracker using LinkedList

Question: Create an ArtifactTracker using LinkedList to store artifact names; add three artifacts, add one at the beginning, remove one artifact, then display the first and last artifacts.

Complete Java Answer

```
import java.util.LinkedList;

class ArtifactTracker {
    public static void main(String[] args) {
        LinkedList<String> artifacts = new LinkedList<>();

        artifacts.add("Ancient Coin");
        artifacts.add("Old Manuscript");
        artifacts.add("Bronze Statue");

        artifacts.addFirst("Royal Sword");
        artifacts.remove("Old Manuscript");

        System.out.println("Artifacts: " + artifacts);
        System.out.println("First artifact: " + artifacts.getFirst());
        System.out.println("Last artifact: " + artifacts.getLast());
    }
}
```

Key points: Uses LinkedList methods add(), addFirst(), remove(), getFirst(), and getLast().

Output:

```
Artifacts: [Royal Sword, Ancient Coin, Bronze Statue]
First artifact: Royal Sword
Last artifact: Bronze Statue
```

Q3. Smart Parking System — Thread Synchronization

Question: Create ParkingArea extending Thread with vehicleNo and shared static availableSlots. parkVehicle() books a slot if available and decreases availableSlots. Synchronize the method and override run().

Complete Java Answer

```
public class Main {
    public static void main(String[] args) {
        ParkingArea v1 = new ParkingArea();
        v1.vehicleNo = "UP11K2026";
        v1.start();

        ParkingArea v2 = new ParkingArea();
        v2.vehicleNo = "UK08K2020";
        v2.start();
    }
}

class ParkingArea extends Thread {
    String vehicleNo;
    static int availableSlots = 1;

    synchronized void parkVehicle(String vehicleNo) {
        if (availableSlots > 0) {
            System.out.println(vehicleNo + " booked a parking slot.");
            availableSlots--;
            System.out.println("Available slots: " + availableSlots);
        } else {
            System.out.println("No parking slot available for " + vehicleNo);
        }
    }

    @Override
    public void run() {
        parkVehicle(vehicleNo);
    }
}
```

Key points: availableSlots is shared by all thread objects. synchronized prevents simultaneous booking of the same slot.

Output:

```
One possible output:
UP11K2026 booked a parking slot.
Available slots: 0
No parking slot available for UK08K2020
(Thread order can vary.)
```

Q4. EV Vehicle — Encapsulation, Constructor and static

Question: Create EVVehicle with private registrationNo, model and batteryPercentage; provide getters/setters, a parameterized constructor, static vehicleCount, and final stationName initialized to "GreenCharge EV Station".

Complete Java Answer

```
public class Main {
    public static void main(String[] args) {
        EVVehicle v1 = new EVVehicle(1, "Nexon", 78);
        EVVehicle v2 = new EVVehicle(2, "Brezza", 50);

        System.out.println("Station: " + EVVehicle.stationName);
        System.out.println(v1.getRegistrationNo() + " "
                + v1.getModel() + " "
                + v1.getBatteryPercentage());
        System.out.println("Total Vehicles: " + EVVehicle.vehicleCount);
    }
}
```

```
class EVVehicle {
    private int registrationNo;
    private String model;
    private int batteryPercentage;

    static int vehicleCount = 0;
    final static String stationName = "GreenCharge EV Station";

    public EVVehicle(int registrationNo, String model,
                     int batteryPercentage) {
        this.registrationNo = registrationNo;
        this.model = model;
        this.batteryPercentage = batteryPercentage;
        vehicleCount++;
    }

    public int getRegistrationNo() {
        return registrationNo;
    }

    public void setRegistrationNo(int registrationNo) {
        this.registrationNo = registrationNo;
    }

    public String getModel() {
        return model;
    }

    public void setModel(String model) {
        this.model = model;
    }

    public int getBatteryPercentage() {
        return batteryPercentage;
    }

    public void setBatteryPercentage(int batteryPercentage) {
        this.batteryPercentage = batteryPercentage;
    }
}
```

Key points: Private fields provide encapsulation. The constructor initializes each object. vehicleCount is shared; stationName is final.

Output:

```
Station: GreenCharge EV Station
1 Nexon 78
Total Vehicles: 2
```

Q5. Smart Home System — Interfaces

Question: Create SecuritySystem with monitorSecurity(), EnergySystem with checkEnergyUsage(), and SmartHome implementing both interfaces. Override both methods with the required messages.

Complete Java Answer

```
public class Main {
    public static void main(String[] args) {
        SmartHome sh = new SmartHome();
        sh.monitorSecurity();
        sh.checkEnergyUsage();
    }
}

interface SecuritySystem {
    void monitorSecurity();
}

interface EnergySystem {
    void checkEnergyUsage();
}

class SmartHome implements SecuritySystem, EnergySystem {

    @Override
    public void monitorSecurity() {
        System.out.println("Security is monitored");
```

```
    }

    @Override
    public void checkEnergyUsage() {
        System.out.println("Energy check done");
    }
}
```

Key points: SmartHome implements two interfaces and provides implementations for both interface methods.

Output:

```
Security is monitored
Energy check done
```

Q6. Online Shopping System — ArrayList

Question: Create ProductDetails with productId, productName and price plus a constructor. Create an ArrayList, add the three products, display them using a for loop, remove one product, and display the updated list.

Complete Java Answer

```
import java.util.ArrayList;

class ProductDetails {
    int productId;
    String productName;
    int price;

    ProductDetails(int productId, String productName, int price) {
        this.productId = productId;
        this.productName = productName;
        this.price = price;
    }

    public String toString() {
        return productId + " " + productName + " " + price;
    }
}

public class Main {
    public static void main(String[] args) {
        ProductDetails p1 = new ProductDetails(1, "Soap", 100);
        ProductDetails p2 = new ProductDetails(2, "Gel", 300);
        ProductDetails p3 = new ProductDetails(3, "Shampoo", 400);

        ArrayList<ProductDetails> products = new ArrayList<>();

        products.add(p1);
        products.add(p2);
        products.add(p3);

        System.out.println("Products:");
        for (ProductDetails p : products) {
            System.out.println(p);
        }

        products.remove(p2);

        System.out.println("After removing one product:");
        for (ProductDetails p : products) {
            System.out.println(p);
        }
    }
}
```

Key points: ArrayList stores ProductDetails objects. toString() makes object output readable. remove(p2) removes the selected product.

Output:

```
Products:
1 Soap 100
2 Gel 300
3 Shampoo 400
After removing one product:
1 Soap 100
3 Shampoo 400
```

Q7. College Event Registration — HashSet

Question: Use a suitable Java collection so duplicate student names are not allowed. Add at least four students, display the total using size(), remove one student, and display all names.

Complete Java Answer

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> students = new HashSet<>();

        students.add("Rahul");
        students.add("Aman");
        students.add("Priya");
        students.add("Neha");

        students.add("Rahul"); // duplicate

        System.out.println("Total students: " + students.size());

        students.remove("Aman");

        System.out.println("Registered students:");
        for (String name : students) {
            System.out.println(name);
        }
    }
}
```

Key points: HashSet does not allow duplicate elements. Iteration order is not guaranteed.

Output:

```
Total students: 4
Registered students:
Rahul
Priya
Neha
(Order may vary.)
```

Q8. Food Order Management — LinkedList

Question: Create a LinkedList of food items. Add Pizza, Burger, Pasta and Sandwich at the beginning, display the first item, remove the last item, and display the remaining items.

Complete Java Answer

```
import java.util.LinkedList;

public class FoodOrder {
    public static void main(String[] args) {
        LinkedList<String> foodItems = new LinkedList<>();

        foodItems.addFirst("Pizza");
        foodItems.addFirst("Burger");
        foodItems.addFirst("Pasta");
        foodItems.addFirst("Sandwich");

        System.out.println("First food item: "
                + foodItems.getFirst());

        foodItems.removeLast();

        System.out.println("Food items:");
        for (String food : foodItems) {
            System.out.println(food);
        }
    }
}
```

Key points: addFirst() inserts at the beginning, getFirst() returns the first item, and removeLast() removes the last item.

Output:

First food item: Sandwich
Food items:
Sandwich
Pasta
Burger